

# 第1章: 開発環境のセットアップ

この章では、書籍管理Webアプリケーションを開発するために必要なツールをすべてインストールし、正しく動作することを確認します。プログラミングが初めての方でも迷わないよう、手順を一つひとつ丁寧に解説します。

## この章で行うこと

「開発環境のセットアップ」とは、プログラムを書いて動かすために必要なソフトウェアをパソコンにインストールする作業です。料理に例えると、実際に料理を始める前に「包丁・まな板・鍋」を揃えるようなものです。

具体的には、以下の4つのツールをインストールします。

| ツール     | 料理で例えると         | なぜ必要か                                                                  |
|---------|-----------------|------------------------------------------------------------------------|
| Node.js | ガスコンロ（調理の火力源）   | プログラムを動かすエンジン。これがないと何も始まりません                                           |
| VS Code | 作業台（まな板と包丁のセット） | プログラムを書くためのエディタ。メモ帳でも書けますが、VS Codeは間違いを指摘してくれたり、コードに色を付けてくれたりする優秀な道具です |
| Git     | レシピノート          | コードの変更履歴を記録するツール。「さっきの状態に戻したい」が簡単にできます                                 |
| ターミナル   | キッチン全体          | すべてのツールを操作するための入口。文字を打ってパソコンに指示を出します                                   |

所要時間の目安：30～60分（ダウンロードの待ち時間を含む）

## 目次

1. Node.js のインストール
2. パッケージマネージャーの理解
3. コードエディタ (VS Code)
4. Git の基礎
5. ターミナル/コマンドラインの基礎
6. 動作確認

# 1. Node.js のインストール

## 1.1 Node.js とは何か

Node.js（ノードジェイエス）は、JavaScript（ジャバスクリプト：Webブラウザ上で動くプログラミング言語）をブラウザの外（サーバーやあなたのPC）で実行できるようにする**ランタイム環境**（Runtime Environment：プログラムを実行するための基盤ソフトウェア）です。

もともとJavaScriptはブラウザの中だけで動く言語でしたが、2009年にNode.jsが登場したことで、サーバーサイド（サーバー側）の処理やコマンドラインツール（CLI：キーボードで文字を打って操作するプログラム）の開発にも使えるようになりました。

**なぜNode.jsが重要なのか：** 現代のWeb開発では、プログラムを書いた後に「TypeScriptをJavaScriptに変換する」「複数のファイルを1つにまとめる」「開発用サーバーを起動する」といった様々な処理が必要です。これらはすべてNode.js上で動きます。つまり、Node.jsは**開発の土台**となるソフトウェアです。

### なぜ Node.js が必要なのか

書籍管理アプリの開発では、以下の場面で Node.js を使います。

| 用途              | 説明                               |
|-----------------|----------------------------------|
| フロントエンド開発サーバー   | React などのフレームワークをローカルで動かす        |
| パッケージ管理         | npm を使ってライブラリをインストールする           |
| ビルドツール          | TypeScript のコンパイル、バンドルなど         |
| バックエンド API サーバー | Express.js など REST API を構築する     |
| データベース操作        | Prisma などの ORM を使ってデータベースとやり取りする |

## 開発ワークフローにおける Node.js の位置づけ



## 1.2 インストール方法

Node.js のインストールには2つの方法があります。

1. 公式サイトから直接インストール — 最もシンプルな方法
2. nvm (Node Version Manager) を使う — 推奨される方法

💡 ヒント: 本チュートリアルでは **nvm** を使ったインストールを強く推奨します。理由は、プロジェクトによって必要な Node.js のバージョンが異なることがあり、nvm を使えばバージョンの切り替えが簡単にできるからです。

## 方法 A: nvm を使ったインストール（推奨）

### Windows の場合

Windows では `nvm-windows` を使います（Linux/Mac 用の nvm とは別のツールです）。

#### 手順 1: nvm-windows のダウンロード

1. `nvm-windows` リリースページ にアクセスします
2. 最新版の `nvm-setup.exe` をダウンロードします
3. ダウンロードしたインストーラーを実行します

#### 手順 2: インストーラーの実行

1. 「I accept the agreement」を選択して「Next」
2. インストール先はデフォルトのまま「Next」  
（通常: C:\Users\<ユーザー名>\AppData\Roaming\nvm）
3. Node.js のシンボリックリンク先もデフォルトのまま「Next」  
（通常: C:\Program Files\nodejs）
4. 「Install」をクリック
5. 「Finish」をクリック

#### 手順 3: Node.js のインストール

新しい PowerShell またはコマンドプロンプトを **管理者**として 開き、以下を実行します。

```
# nvm が正しくインストールされたか確認
nvm version

# インストール可能な Node.js のバージョン一覧を確認
nvm list available

# LTS（長期サポート）版をインストール（推奨）
nvm install lts

# インストールした Node.js を使用する
nvm use lts

# 確認
node -v
npm -v
```

 **注意:** Windows では `nvm install` と `nvm use` コマンドの実行に **管理者権限** が必要です。  
PowerShell を右クリックして「管理者として実行」を選んでください。

### Mac の場合

#### 手順 1: nvm のインストール

ターミナル (Terminal.app) を開いて以下を実行します。

```
# nvm インストールスクリプトを実行
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh | bash
```

## 手順 2: シェルの再起動

インストール後、ターミナルを一度閉じて再度開きます。または以下を実行します。

```
# zsh の場合 (macOS のデフォルト)
source ~/.zshrc

# bash の場合
source ~/.bashrc
```

## 手順 3: Node.js のインストール

```
# nvm が使えるか確認
nvm --version

# LTS 版をインストール
nvm install --lts

# 確認
node -v
npm -v
```

💡 **ヒント:** Mac で Homebrew を使っている場合は `brew install nvm` でもインストールできますが、公式のインストールスクリプトを使う方が確実です。

## Linux (Ubuntu/Debian) の場合

```
# 必要なパッケージをインストール
sudo apt update
sudo apt install curl -y

# nvm インストールスクリプトを実行
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh | bash

# シェル設定を再読み込み
source ~/.bashrc

# nvm が使えるか確認
nvm --version

# LTS 版をインストール
nvm install --lts

# デフォルトバージョンとして設定
nvm alias default node

# 確認
node -v
npm -v
```

## 方法 B: 公式サイトから直接インストール

nvm を使わない場合は、公式サイトから直接インストールできます。

1. [Node.js 公式サイト](#) にアクセスします
2. LTS (推奨版) をクリックしてダウンロードします
3. ダウンロードしたインストーラーを実行します

```
Windows: .msi ファイルを実行 → 画面の指示に従う
Mac:     .pkg ファイルを実行 → 画面の指示に従う
Linux:   公式リポジトリを追加してインストール (下記参照)
```

Linux (Ubuntu/Debian) で公式リポジトリを使う場合:

```
# NodeSource リポジトリを追加 (Node.js 20.x の場合)
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -

# Node.js をインストール
sudo apt install -y nodejs

# 確認
node -v
npm -v
```

## 1.3 バージョン確認

インストールが完了したら、ターミナル（Windows は PowerShell、Mac/Linux はターミナル）で以下を実行して確認します。

```
# Node.js のバージョン確認
node -v
# 出力例: v20.11.0

# npm のバージョン確認
npm -v
# 出力例: 10.2.4

# Node.js が正しく動作するかテスト
node -e "console.log('Hello, Node.js!')"
# 出力: Hello, Node.js!
```

 **ヒント:** 本チュートリアルでは **Node.js 18.x 以上** を推奨します。**node -v** で表示されるバージョンが **v18.0.0** 以上であることを確認してください。

## 1.4 トラブルシューティング

「node」コマンドが見つからない

'node' は、内部コマンドまたは外部コマンド、  
操作可能なプログラムまたはバッチ ファイルとして認識されていません。

原因: Node.js へのパスが環境変数 PATH に追加されていません。

解決方法（Windows）：

```
# 1. 現在の PATH を確認
$env:PATH -split ";"

# 2. Node.js のインストール先を確認
# 通常は以下のいずれか
# - C:\Program Files\nodejs\
# - C:\Users\<ユーザー名>\AppData\Roaming\nvm\<バージョン>\

# 3. 環境変数を設定 (システムのプロパティ → 環境変数 → PATH に追加)
# または PowerShell で一時的に追加:
$env:PATH += ";C:\Program Files\nodejs\"

# 4. ターミナルを再起動して再度確認
node -v
```

解決方法 (Mac/Linux) :

```
# 1. Node.js のインストール先を確認
which node
# または
ls /usr/local/bin/node

# 2. nvm を使っている場合、シェル設定ファイルに以下が追加されているか確認
cat ~/.bashrc | grep NVM
# または
cat ~/.zshrc | grep NVM

# 以下のような行があるはず:
# export NVM_DIR="$HOME/.nvm"
# [ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"

# 3. なければ手動で追加
echo 'export NVM_DIR="$HOME/.nvm"' >> ~/.zshrc
echo '[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"' >> ~/.zshrc
source ~/.zshrc
```



## Node.js のバージョンが古い

```
# 現在のバージョン確認
node -v
# 出力例: v14.17.0 ← 古すぎる

# nvm を使っている場合
nvm install --lts
nvm use --lts

# nvm を使っていない場合
# → 公式サイトから最新 LTS 版をダウンロードして再インストール
```

## nvm コマンドが見つからない (Mac/Linux)

```
# シェル設定ファイルを確認
# zsh の場合
cat ~/.zshrc | grep -A2 NVM

# 設定がなければ追加
cat >> ~/.zshrc << 'EOF'
export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"
[ -s "$NVM_DIR/bash_completion" ] && \. "$NVM_DIR/bash_completion"
EOF

# 設定を反映
source ~/.zshrc
```

## npm install 時に EACCES エラー (Mac/Linux)

```
npm ERR! Error: EACCES: permission denied
```

```
# nvm を使っていればこのエラーは通常発生しません
# グローバルインストール先を変更する方法:

mkdir ~/.npm-global
npm config set prefix '~/.npm-global'
echo 'export PATH=~/.npm-global/bin:$PATH' >> ~/.bashrc
source ~/.bashrc
```

## 2. パッケージマネージャーの理解

### 2.1 パッケージマネージャーとは

パッケージマネージャーは、プロジェクトで使うライブラリ（パッケージ）のインストール、更新、削除を管理するツールです。たとえば、「React」や「Express」などのライブラリをコマンド一つでインストールできます。



### 2.2 npm vs yarn vs pnpm 比較

| 特徴         | npm                          | yarn             | pnpm           |
|------------|------------------------------|------------------|----------------|
| 開発元        | npm, Inc. (GitHub/Microsoft) | Meta (旧Facebook) | コミュニティ         |
| Node.js 同梱 | はい (デフォルト)                   | いいえ (別途インストール)   | いいえ (別途インストール) |
| インストール速度   | 普通                           | 速い               | 最も速い           |
| ディスク使用量    | 多い                           | 多い               | 少ない (ハードリンク)   |
| ロックファイル    | package-lock.json            | yarn.lock        | pnpm-lock.yaml |
| ワークスペース対応  | v7 以降対応                      | 対応               | 対応             |
| 学習コスト      | 低い                           | 低い               | やや高い           |
| 初心者おすすめ度   | ★★★★★                        | ★★★★☆            | ★★★☆☆          |

💡 **ヒント:** 本チュートリアルでは **npm** を使用します。Node.js をインストールすると自動的に npm もインストールされるため、追加のセットアップが不要です。

## 2.3 package.json の理解

**package.json** はプロジェクトの「設計図」のようなファイルです。プロジェクトの名前、バージョン、使用するライブラリなどの情報がすべて記載されています。

```
{
  "name": "book-management-app",
  "version": "1.0.0",
  "description": "書籍管理 Web アプリケーション",
  "main": "index.js",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "start": "node server.js",
    "test": "vitest"
  },
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "express": "^4.18.2"
  },
  "devDependencies": {
    "vite": "^5.0.0",
    "vitest": "^1.0.0",
    "typescript": "^5.3.0"
  }
}
```

各フィールドの意味:

| フィールド                  | 説明                                      |
|------------------------|-----------------------------------------|
| <b>name</b>            | プロジェクト名（小文字、ハイフン区切り）                    |
| <b>version</b>         | プロジェクトのバージョン（セマンティックバージョンング）            |
| <b>description</b>     | プロジェクトの説明文                              |
| <b>scripts</b>         | <b>npm run &lt;名前&gt;</b> で実行できるコマンドの定義 |
| <b>dependencies</b>    | 本番環境でも必要なライブラリ                          |
| <b>devDependencies</b> | 開発時のみ必要なライブラリ（テストツール、ビルドツール等）           |

## バージョン指定の記号

| 記号             | 意味                       | 例                                 |
|----------------|--------------------------|-----------------------------------|
| <code>^</code> | メジャーバージョン固定（マイナー・パッチは最新） | <code>^18.2.0</code> → 18.x.x の最新 |
| <code>~</code> | メジャー・マイナー固定（パッチのみ最新）     | <code>~18.2.0</code> → 18.2.x の最新 |
| なし             | 完全固定                     | <code>18.2.0</code> → 18.2.0 のみ   |
| <code>*</code> | 任意のバージョン（非推奨）            | <code>*</code> → 何でもOK            |

## 2.4 node\_modules とは

`node_modules` フォルダは、`npm install` を実行したときにダウンロードされるライブラリの実体が格納される場所です。

```
my-project/
├─ node_modules/      ← ライブラリの実体（数千ファイル）
│   ├─ react/
│   ├─ react-dom/
│   ├─ express/
│   └─ ... 数百のパッケージ
├─ package.json        ← 依存関係の定義
├─ package-lock.json   ← バージョンの完全固定
└─ src/                ← あなたのコード
```

⚠ **注意:** `node_modules` フォルダは非常に大きくなります（数百MB になることもあります）。`Git` にはコミットしないでください。後の章で設定する `.gitignore` ファイルで除外します。

💡 **ヒント:** `node_modules` を削除してしまっても、`npm install` を実行すれば `package.json` と `package-lock.json` の情報をもとにすべて復元できます。

## 2.5 基本的な npm コマンド

```
# プロジェクトの初期化 (package.json を作成)
npm init -y

# パッケージのインストール (dependencies に追加)
npm install react

# 開発用パッケージのインストール (devDependencies に追加)
npm install --save-dev vitest

# package.json に記載された全パッケージをインストール
npm install

# パッケージの削除
npm uninstall react

# スクリプトの実行
npm run dev

# インストール済みパッケージの一覧
npm list --depth=0

# パッケージの更新確認
npm outdated
```

## 2.6 パッケージ管理のフロー



## 3. コードエディタ (VS Code)

### 3.1 VS Code とは

Visual Studio Code (VS Code) は、Microsoft が開発した **無料** のコードエディタです。軽量でありながら強力な機能を持ち、豊富な拡張機能によってあらゆるプログラミング言語に対応できます。Web 開発において最も人気のあるエディタの一つです。

### 3.2 インストール手順

#### Windows

1. [VS Code 公式サイト](#) にアクセス
2. 「Download for Windows」をクリック
3. ダウンロードした **.exe** ファイルを実行
4. インストールオプション: - 「PATHへの追加」にチェックを入れる（重要） - 「エクスプローラーのファイルコンテキストメニューに "Code で開く" を追加」にチェックを入れる（便利） - 「エクスプローラーのディレクトリコンテキストメニューに "Code で開く" を追加」にチェックを入れる（便利）
5. 「インストール」をクリック

⚠ **注意:** 「PATHへの追加」にチェックを入れ忘れると、ターミナルから **code** コマンドで VS Code を起動できません。忘れた場合は再インストールしてください。

#### Mac

1. [VS Code 公式サイト](#) にアクセス
2. 「Download for Mac」をクリック
3. ダウンロードした **.zip** ファイルを展開
4. **Visual Studio Code.app** を「アプリケーション」フォルダにドラッグ
5. **code** コマンドを使えるようにする: - VS Code を起動 - **Cmd + Shift + P** でコマンドパレットを開く - 「Shell Command: Install 'code' command in PATH」を選択

## Linux (Ubuntu/Debian)

```
# Microsoft の GPG キーとリポジトリを追加
wget -qO- https://packages.microsoft.com/keys/microsoft.asc | gpg --dearmor > packages.mic
sudo install -D -o root -g root -m 644 packages.microsoft.gpg /etc/apt/keyrings/packages.m
echo "deb [arch=amd64,arm64,armhf signed-by=/etc/apt/keyrings/packages.microsoft.gpg] http
rm -f packages.microsoft.gpg

# インストール
sudo apt update
sudo apt install code -y

# 確認
code --version
```

### 3.3 推奨拡張機能

VS Code のサイドバーから拡張機能アイコン（四角が4つのアイコン）をクリックし、以下の拡張機能を検索してインストールしてください。

#### 必須の拡張機能

| 拡張機能名                                  | ID                                | 用途                               | 重要度 |
|----------------------------------------|-----------------------------------|----------------------------------|-----|
| Japanese Language Pack                 | MS-CEINTL.vscode-language-pack-ja | VS Code の日本語化                    | 必須  |
| ESLint                                 | dbaeumer.vscode-eslint            | JavaScript/TypeScript のコード品質チェック | 必須  |
| Prettier - Code formatter              | esbenp.prettier-vscode            | コードの自動フォーマット                     | 必須  |
| ES7+ React/Redux/React-Native snippets | dsznajder.es7-react-js-snippets   | React のコードスニペット                  | 必須  |



## 強く推奨する拡張機能

| 拡張機能名               | ID                                              | 用途                    | 重要度 |
|---------------------|-------------------------------------------------|-----------------------|-----|
| TypeScript Importer | <code>pmneo.tsimporter</code>                   | import 文の自動補完         | 推奨  |
| Auto Rename Tag     | <code>formulahendry.auto-rename-tag</code>      | HTML/JSX タグの自動リネーム    | 推奨  |
| Path Intellisense   | <code>christian-kohler.path-intellisense</code> | ファイルパスの自動補完           | 推奨  |
| GitLens             | <code>eamodio.gitlens</code>                    | Git の履歴・変更を可視化        | 推奨  |
| Error Lens          | <code>usernamehw.errorlens</code>               | エラーをエディタ上にインライン表示     | 推奨  |
| Thunder Client      | <code>rangav.vscode-thunder-client</code>       | API テスト (Postman の代替) | 推奨  |

## あると便利な拡張機能

| 拡張機能名                  | ID                                                      | 用途              | 重要度 |
|------------------------|---------------------------------------------------------|-----------------|-----|
| Material Icon Theme    | <code>PKief.material-icon-theme</code>                  | ファイルアイコンの見た目を改善 | 任意  |
| indent-rainbow         | <code>oderwat.indent-rainbow</code>                     | インデントを色分け表示     | 任意  |
| Bracket Pair Color DLW | <code>BracketPairColorDLW.bracket-pair-color-dlw</code> | 対応する括弧を色分け      | 任意  |
| Code Spell Checker     | <code>streetsidesoftware.code-spell-checker</code>      | スペルチェック         | 任意  |

💡 ヒント: コマンドラインから一括インストールすることもできます: `bash code --install-extension MS-CEINTL.vscode-language-pack-ja code --install-extension dbaeumer.vscode-eslint code --install-extension esbenp.prettier-vscode code --install-extension dsznajder.es7-react-js-snippets code --install-extension eamodio.gitlens code --install-extension usernamehw.errorlens`

## 3.4 基本的な使い方

### キーボードショートカット（Windows / Mac）

| 操作           | Windows          | Mac                |
|--------------|------------------|--------------------|
| コマンドパレット     | Ctrl + Shift + P | Cmd + Shift + P    |
| ファイル検索       | Ctrl + P         | Cmd + P            |
| 全文検索         | Ctrl + Shift + F | Cmd + Shift + F    |
| ターミナルの表示/非表示 | Ctrl + `         | Cmd + `            |
| サイドバーの表示/非表示 | Ctrl + B         | Cmd + B            |
| 行の複製         | Shift + Alt + ↓  | Shift + Option + ↓ |
| 行の移動         | Alt + ↑/↓        | Option + ↑/↓       |
| 複数カーソル       | Ctrl + Alt + ↑/↓ | Cmd + Option + ↑/↓ |
| 名前の一括変更      | F2               | F2                 |
| 定義へ移動        | F12              | F12                |
| 保存           | Ctrl + S         | Cmd + S            |
| 全保存          | Ctrl + K, S      | Cmd + Option + S   |

### フォルダを開いて作業を始める

```
# ターミナルから VS Code でフォルダを開く
code /path/to/my-project

# 現在のフォルダを VS Code で開く
code .
```

### 推奨する VS Code 設定

VS Code で **Ctrl + Shift + P**（Mac: **Cmd + Shift + P**）を押して「Preferences: Open Settings (JSON)」を選択し、以下の設定を追加します。

```
{
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.tabSize": 2,
  "editor.wordWrap": "on",
  "editor.minimap.enabled": false,
  "editor.bracketPairColorization.enabled": true,
  "editor.guides.bracketPairs": "active",
  "files.autoSave": "onFocusChange",
  "terminal.integrated.defaultProfile.windows": "Git Bash",
  "emmet.includeLanguages": {
    "javascript": "javascriptreact"
  }
}
```

| 設定                                  | 効果                                |
|-------------------------------------|-----------------------------------|
| <code>formatOnSave</code>           | ファイル保存時に自動フォーマット                  |
| <code>defaultFormatter</code>       | Prettier をデフォルトフォーマッターに設定         |
| <code>tabSize: 2</code>             | インデント幅を2スペースに設定                   |
| <code>wordWrap</code>               | 長い行を折り返して表示                       |
| <code>autoSave</code>               | フォーカスが外れたら自動保存                    |
| <code>defaultProfile.windows</code> | Windows のデフォルトターミナルを Git Bash に変更 |

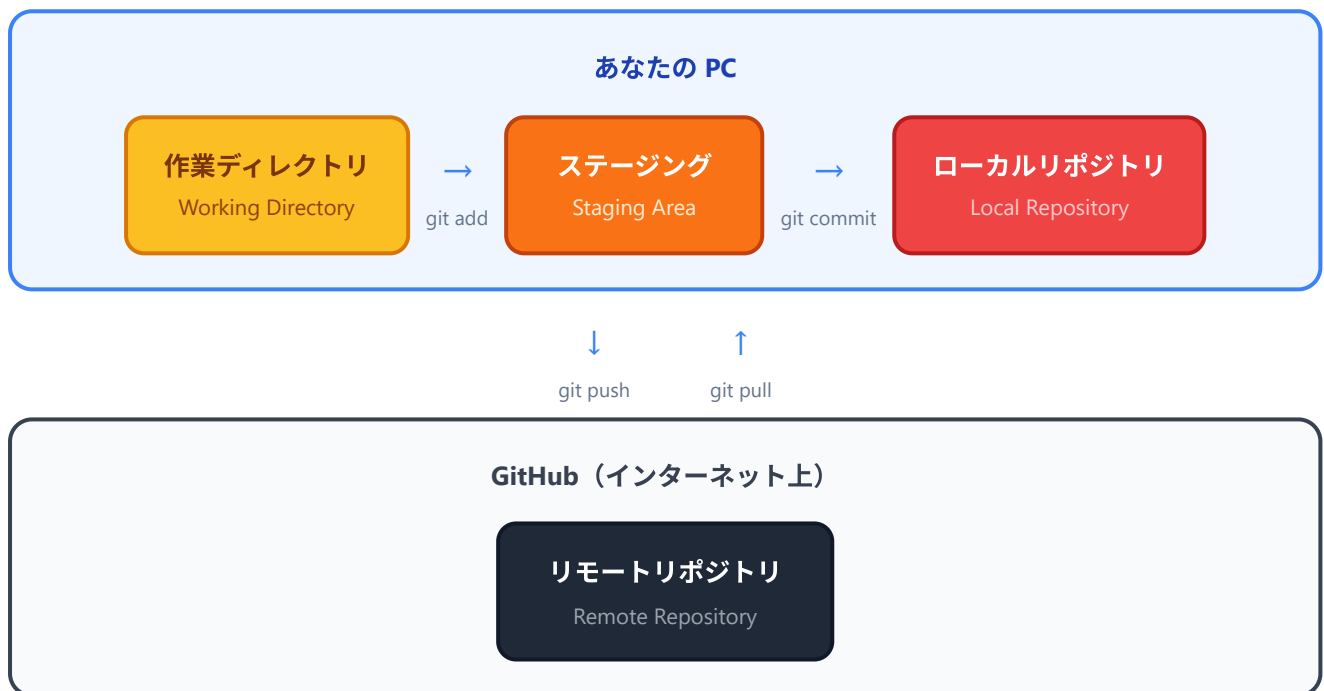
## 4. Git の基礎

### 4.1 Git とは

Git はバージョン管理システムです。ファイルの変更履歴を記録し、過去の状態に戻したり、複数人で同時に開発したりすることを可能にします。

## なぜ Git が必要なのか

| 状況            | Git なし        | Git あり                          |
|---------------|---------------|---------------------------------|
| コードを壊してしまった   | 元に戻せない        | <code>git checkout</code> で復元可能 |
| 誰がいつ変更したか知りたい | わからない         | <code>git log</code> で確認可能      |
| 新機能を試したい      | メインのコードを壊すリスク | ブランチで安全に実験可能                    |
| 複数人で開発        | ファイルの上書き事故    | マージ機能で安全に統合                     |
| コードを公開・共有したい  | USBやメールで送る    | GitHub にプッシュ                    |



## 4.2 Git のインストール

### Windows

1. [Git for Windows](#) にアクセス
2. インストーラーをダウンロードして実行
3. インストールオプション（以下のデフォルト推奨設定を確認）：
  - **Default editor**: 「Use Visual Studio Code as Git's default editor」を選択
  - **Adjusting your PATH**: 「Git from the command line and also from 3rd-party software」を選択
  - **Line ending conversions**: 「Checkout Windows-style, commit Unix-style line endings」を選択
  - その他はデフォルトのまま「Next」を押してインストール

💡 **ヒント**: Git for Windows をインストールすると **Git Bash** というターミナルも一緒にインストールされます。Linux/Mac と同じコマンドが使えるので非常に便利です。

## Mac

```
# Xcode Command Line Tools に含まれている Git を使う方法
xcode-select --install

# または Homebrew を使う方法（推奨：最新版が使える）
brew install git
```

## Linux (Ubuntu/Debian)

```
sudo apt update
sudo apt install git -y
```

## バージョン確認

```
git --version
# 出力例: git version 2.43.0
```

## 4.3 Git の初期設定

インストール後、ユーザー名とメールアドレスを設定します。これはコミット（変更の記録）に記録される情報です。

```
# ユーザー名を設定
git config --global user.name "あなたの名前"

# メールアドレスを設定 (GitHub アカウントと同じものを推奨)
git config --global user.email "your-email@example.com"

# デフォルトブランチ名を main に設定
git config --global init.defaultBranch main

# 設定の確認
git config --global --list
```

## 4.4 基本コマンド

### リポジトリの作成と基本操作

Git の基本フローを「ファイル作成 → ステージ → コミット」の流れで体験します。各コマンドが何をしているか1行ずつ解説します。

```

# -----
# (1) 新しい作業フォルダを作って、その中に移動する
# -----
# mkdir = "make directory"。フォルダを新規作成するコマンド。
mkdir my-project

# cd = "change directory"。カレントディレクトリ（今いる場所）を移動。
cd my-project

# -----
# (2) Git リポジトリを初期化する
# -----
# git init は「このフォルダをGit管理下にする」コマンド。
# 隠しフォルダ .git/ が作られ、ここに変更履歴が全部保存される。
git init
# ▼ 期待する出力
# Initialized empty Git repository in /path/to/my-project/.git/

# -----
# (3) サンプルファイルを1個作る
# -----
# echo "... " は文字列をそのまま出力するコマンド。
# > README.md は「出力先をファイルに切り替える」リダイレクト演算子。
# 結果として「README.md という名前のファイルに "# My Project" と書く」操作になる。
echo "# My Project" > README.md

# -----
# (4) 現在のリポジトリの状態を確認する
# -----
# git status は「いま何が変わってる？」を教えてくれる。最頻出コマンド。
git status
# ▼ 期待する出力（要約）
# On branch main
# No commits yet
# Untracked files:
#   (use "git add <file>..." to include in what will be committed)
#       README.md
# nothing added to commit but untracked files present (use "git add" to track)
# ▼ 「Untracked」= まだGitに認識されていない新規ファイル、の意味。

# -----
# (5) ファイルを「ステージングエリア」に追加
# -----
# Gitは2段階で変更を記録する。

```

```

# ①ステージング (git add) ← どの変更を記録するかを選ぶ
# ②コミット (git commit) ← 選んだ変更をスナップショットとして記録
# こうすることで「複数の変更のうち一部だけまとめてコミット」ができる。
git add README.md          # 特定ファイルだけ追加
# git add .                  # ピリオドで「カレントの全変更を追加」

# -----
# (6) コミットする = スナップショットを履歴に記録
# -----
# -m "... " はコミットメッセージ。なぜこの変更をしたかを短文で残す。
# 必ず引用符で囲む。日本語OK。
git commit -m "最初のコミット: READMEを追加"
# ▼ 期待する出力 (要約)
# [main (root-commit) 1a2b3c4] 最初のコミット: READMEを追加
# 1 file changed, 1 insertion(+)
# create mode 100644 README.md

# -----
# (7) コミット履歴を見る
# -----
# git log だけだと1コミットあたり数行使うので画面が長くなる。
# --oneline で1行に圧縮表示できる。
git log
git log --oneline
# ▼ 出力例 (git log --oneline)
# 1a2b3c4 (HEAD -> main) 最初のコミット: READMEを追加

```

## リモートリポジトリとの連携

```
# -----
# (1) リモートリポジトリ「origin」を登録
# -----
# git remote add <名前> <URL>: 別の場所にあるGitリポジトリに別名 (origin) を付ける。
# origin は慣習的に「メインのリモート」を指す名前。GitHubで作ったリポジトリのURLを使う。
git remote add origin https://github.com/ユーザー名/リポジトリ名.git

# -----
# (2) ローカルのコミットをGitHubに送る（プッシュ）
# -----
# git push <リモート名> <ブランチ名>
# -u は「次回からは git push だけで origin main に送るように覚えておく」設定。
git push -u origin main
# ▼ 出力例
# Enumerating objects: 3, done.
# Counting objects: 100% (3/3), done.
# Writing objects: 100% (3/3), 234 bytes | 234.00 KiB/s, done.
# Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
# To https://github.com/yourname/my-project.git
# * [new branch]      main -> main
# branch 'main' set up to track 'origin/main'.

# -----
# (3) GitHubの最新コミットをローカルに取り込む（プル）
# -----
# 他の開発者やGitHub上で行われた変更を、自分のローカルに反映する。
git pull origin main
```



## よく使うコマンドの一覧

| コマンド                                    | 説明               |
|-----------------------------------------|------------------|
| <code>git init</code>                   | リポジトリを初期化        |
| <code>git status</code>                 | 変更状態を確認          |
| <code>git add &lt;ファイル&gt;</code>       | ステージングエリアに追加     |
| <code>git add .</code>                  | すべての変更をステージに追加   |
| <code>git commit -m "メッセージ"</code>      | 変更をコミット          |
| <code>git log --oneline</code>          | コミット履歴を簡潔に表示     |
| <code>git diff</code>                   | 変更内容の差分を表示       |
| <code>git branch</code>                 | ブランチ一覧を表示        |
| <code>git checkout -b &lt;名前&gt;</code> | 新しいブランチを作成して切り替え |
| <code>git push</code>                   | リモートにプッシュ        |
| <code>git pull</code>                   | リモートからプル         |
| <code>git clone &lt;URL&gt;</code>      | リポジトリをコピー        |

## 4.5 GitHub アカウントの作成

1. [GitHub](#) にアクセス
2. 「Sign up」をクリック
3. メールアドレス、パスワード、ユーザー名を入力
4. メール認証を完了
5. 無料プラン（Free）を選択

## SSH キーの設定（推奨）

パスワード入力なしで GitHub にプッシュできるように SSH キーを設定します。

```
# SSH キーを生成
ssh-keygen -t ed25519 -C "your-email@example.com"
# Enter を3回押す（デフォルト設定でOK）

# 公開鍵を表示
cat ~/.ssh/id_ed25519.pub
# 出力された文字列をすべてコピー
```

GitHub での設定:

1. GitHub にログイン
2. 右上のアイコン → 「Settings」
3. 左メニュー「SSH and GPG keys」
4. 「New SSH key」をクリック
5. Title: 任意の名前（例: "My PC"）
6. Key: コピーした公開鍵を貼り付け
7. 「Add SSH key」をクリック

```
# 接続テスト
ssh -T git@github.com
# 出力: Hi ユーザー名! You've successfully authenticated...
```

## 4.6 Git トラブルシューティング

### push 時に認証エラー

```
remote: Support for password authentication was removed
```

原因: GitHub はパスワード認証を廃止しました。

解決方法: SSH キーを設定するか、Personal Access Token を使用します。上記の SSH キーの設定を参照してください。

### 改行コードの警告 (Windows)

```
warning: LF will be replaced by CRLF
```

解決方法:

```
# この警告を抑制する
git config --global core.autocrlf true
```

### コミットメッセージのエディタが vim で困る

```
# VS Code をデフォルトエディタに設定
git config --global core.editor "code --wait"
```

## 5. ターミナル/コマンドラインの基礎

### 5.1 ターミナルとは

ターミナル（コマンドライン）は、テキストでコンピュータに指示を出すためのツールです。GUI（マウスでクリックする画面）では操作が複雑になるような作業も、コマンド一つで実行できます。

Web 開発ではターミナルを頻繁に使うため、基本的なコマンドを覚えておくことが重要です。

#### ターミナルの画面はこんな見た目

初めてターミナルを開くと、こんな画面が出てきます（黒や白の背景に、緑や白の文字）。

```
yuya@MyPC MINGW64 ~/Desktop
$ _
```

各部分の意味は次のとおりです。

| 部分         | 例         | 意味                                     |
|------------|-----------|----------------------------------------|
| ユーザー名      | yuya      | ログイン中のユーザーの名前                          |
| PC名        | MyPC      | あなたのコンピュータの名前                          |
| シェル種別      | MINGW64   | Git Bash の場合に表示される識別子。PowerShellなら別の表示 |
| カレントディレクトリ | ~/Desktop | 「いま自分がいる場所」。~ はホームフォルダ                 |
| プロンプト      | \$ または >  | 「ここからコマンドを打ってね」という合図                   |
| カーソル       | _（点滅）     | 入力位置を示す光る四角                            |

「カレントディレクトリ」って何？：いまターミナルが「自分の現在地」として認識しているフォルダのことです。pwd コマンドで確認でき、cd コマンドで移動できます。Windowsのエクスプローラーで「現在開いているフォルダ」と同じ概念です。

#### コマンドを「実行する」とは？

たとえば pwd というコマンドを実行する手順は以下のとおりです。

1. プロンプト \$ の右側にカーソルが点滅していることを確認
2. キーボードで pwd と打つ
3. Enterキー を押す
4. すぐ下の行に結果（出力）が表示される
5. その下に新しいプロンプトが現れて、次のコマンドを待つ

## ▼ 実行例:

```
$ pwd
/c/Users/yuya/Desktop
$ _
```

ここで `pwd` の右下に表示された `/c/Users/yuya/Desktop` が、コマンドの**実行結果**です。これは「いまDesktopフォルダにいるよ」という意味です。

## 行頭の `$` や `>` はコピーしない

このチュートリアル中で出てくるサンプルコマンドの中には、行頭に `$` が付いているものがあります。

```
$ node -v
```

この `$` は「これはターミナルに打つコマンドですよ」という印で、**実際には打ちません**。打つのは `$` より右の `node -v` だけです。同じく PowerShell のサンプルで `PS C:\>` のような表示があったら、それも飾りです。

**慣習:** `$` は一般ユーザー、`#` は管理者権限ユーザー用のプロンプトとして使い分けことがあります。本書では混乱を避けるため、コマンドだけを書いて `$` は省略するスタイルを基本にしています。

## 5.2 Windows のターミナル選択肢

Windows には複数のターミナルがあります。

| ターミナル            | 特徴                            | 推奨度   |
|------------------|-------------------------------|-------|
| Git Bash         | Linux/Mac と同じコマンドが使える。Git に同梱 | ★★★★★ |
| Windows Terminal | Microsoft 公式。複数のシェルをタブで使い分け可能 | ★★★★☆ |
| PowerShell       | Windows 標準。独自のコマンド体系          | ★★★☆☆ |
| コマンドプロンプト (cmd)  | 旧来の Windows ターミナル。機能が限定的      | ★★☆☆☆ |

 **ヒント:** 本チュートリアルでは **Git Bash** の使用を推奨します。オンラインの記事やドキュメントの多くは Linux/Mac のコマンドで書かれており、Git Bash ならそれらをそのまま実行できます。

### Git Bash の起動方法

- スタートメニューで「Git Bash」と検索して起動
- 任意のフォルダで右クリック → 「Git Bash Here」
- VS Code のターミナルを Git Bash に設定（前述の設定で対応済み）

## PowerShell と Git Bash のコマンド比較

| 操作        | Git Bash / Mac / Linux        | PowerShell                                     |
|-----------|-------------------------------|------------------------------------------------|
| 現在のフォルダ表示 | <code>pwd</code>              | <code>pwd</code> または <code>Get-Location</code> |
| ファイル一覧    | <code>ls</code>               | <code>ls</code> または <code>Get-ChildItem</code> |
| フォルダ移動    | <code>cd folder</code>        | <code>cd folder</code>                         |
| フォルダ作成    | <code>mkdir folder</code>     | <code>mkdir folder</code>                      |
| ファイル削除    | <code>rm file.txt</code>      | <code>Remove-Item file.txt</code>              |
| フォルダ削除    | <code>rm -rf folder</code>    | <code>Remove-Item -Recurse folder</code>       |
| ファイル内容表示  | <code>cat file.txt</code>     | <code>Get-Content file.txt</code>              |
| テキスト検索    | <code>grep "text" file</code> | <code>Select-String "text" file</code>         |
| 環境変数表示    | <code>echo \$PATH</code>      | <code>\$env:PATH</code>                        |
| 画面クリア     | <code>clear</code>            | <code>cls</code> または <code>Clear-Host</code>   |

### 5.3 基本コマンド一覧

以下は、開発で頻繁に使うコマンドです（Git Bash / Mac / Linux 共通）。

## ファイル・フォルダ操作

```
# 現在のフォルダ（ディレクトリ）を表示
pwd
# 出力例: /c/Users/yuya/Desktop/education

# ファイル・フォルダの一覧を表示
ls
# 出力例: Documents Downloads Desktop

# 隠しファイルも含めた詳細一覧
ls -la
# 出力例:
# drwxr-xr-x  5 yuya  staff  160  1  1 12:00 .
# drwxr-xr-x  3 yuya  staff   96  1  1 12:00 ..
# -rw-r--r--  1 yuya  staff   50  1  1 12:00 .gitignore
# -rw-r--r--  1 yuya  staff  200  1  1 12:00 package.json

# フォルダの移動
cd Documents      # Documents フォルダに移動
cd ..             # 一つ上のフォルダに移動
cd ~              # ホームフォルダに移動
cd /c/Users/yuya  # 絶対パスで移動 (Git Bash の場合)

# フォルダの作成
mkdir my-project      # 単一フォルダ作成
mkdir -p src/components/ui  # 入れ子フォルダも一度に作成

# ファイルの作成
touch index.html      # 空のファイルを作成
echo "Hello" > hello.txt  # 内容を指定してファイル作成

# ファイルのコピー
cp file.txt file-backup.txt  # ファイルをコピー
cp -r src/ src-backup/      # フォルダごとコピー

# ファイルの移動・リネーム
mv old-name.txt new-name.txt  # ファイル名変更
mv file.txt Documents/        # ファイルをフォルダに移動

# ファイルの削除
rm file.txt              # ファイルを削除
rm -rf node_modules/     # フォルダを中身ごと削除
```

 **注意:** `rm -rf` は確認なしに完全削除されます。特に `rm -rf /` や `rm -rf ~` のような広範囲の削除は絶対に実行しないでください。取り返しがつきません。

## ファイル内容の表示

```
# ファイルの内容を表示
cat package.json

# 長いファイルをページ単位で表示
less README.md
# (q キーで終了)

# ファイルの先頭を表示
head -n 10 server.js      # 最初の10行

# ファイルの末尾を表示
tail -n 10 server.log     # 最後の10行
```

## その他の便利なコマンド

```
# 画面をクリア
clear

# コマンドの履歴を表示
history

# コマンドの場所を確認
which node
# 出力例: /home/yuya/.nvm/versions/node/v20.11.0/bin/node

# ファイル・フォルダを検索
find . -name "*.js"      # 現在のフォルダ以下の .js ファイルを検索

# テキストを検索
grep "import" src/App.tsx # ファイル内のテキストを検索
grep -r "TODO" src/       # フォルダ内を再帰的に検索
```

## 5.4 パス（Path）の基本

```
# 絶対パス: ルートから始まるフルパス
/c/Users/yuya/Desktop/education/my-project/src/App.tsx

# 相対パス: 現在のフォルダからの相対位置
./src/App.tsx           # 現在のフォルダの src フォルダ内
../other-project/       # 一つ上のフォルダの other-project
../../                 # 二つ上のフォルダ

# 特殊なパス
~                        # ホームフォルダ (/c/Users/yuya)
.                        # 現在のフォルダ
..                       # 一つ上のフォルダ
```

## 5.5 ターミナルのトラブルシューティング

### コマンドが見つからない

```
bash: some-command: command not found
```

#### 確認手順:

```
# コマンドがインストールされているか確認
which some-command

# PATH を確認
echo $PATH

# PATH にコマンドの場所が含まれていない場合
# → 該当ツールを再インストールするか、PATH を設定する
```

### 日本語が文字化けする（Windows）

```
# Git Bash の場合、以下を ~/.bashrc に追加
export LANG=ja_JP.UTF-8
```



## Tab 補完を活用する

```
# ファイル名やフォルダ名を途中まで入力して Tab キーを押す
cd Doc<Tab>
# → cd Documents/ に自動補完される

# 候補が複数ある場合は Tab を2回押すと一覧が表示される
cd D<Tab><Tab>
# → Desktop/ Documents/ Downloads/
```

## 6. 動作確認

すべてのツールが正しくインストールされたか、以下のチェックリストで確認しましょう。

### 6.1 確認コマンドの実行

ターミナル（Git Bash 推奨）を開いて、以下のコマンドを順番に実行してください。

```
echo "=== 開発環境チェック ==="
echo ""

echo "1. Node.js:"
node -v
echo ""

echo "2. npm:"
npm -v
echo ""

echo "3. Git:"
git --version
echo ""

echo "4. VS Code:"
code --version
echo ""

echo "=== チェック完了 ==="
```

▼ 期待する実行結果（バージョンの数字は時期によって変わります）：

```
=== 開発環境チェック ===
```

```
1. Node.js:  
v20.11.0  
  
2. npm:  
10.2.4  
  
3. Git:  
git version 2.43.0.windows.1  
  
4. VS Code:  
1.85.1  
929bacba01ef7d04dac26da25e5dafdc4d039aaf  
x64  
  
=== チェック完了 ===
```

#### ▼ 結果の読み方:

- `node -v` の `v` は version の v。続く数字が Node.js の **メジャー.マイナー.パッチ** バージョン
- `npm -v` は数字のみ。10.x なら本書の手順で問題なく動作する
- `git --version` の末尾 `.windows.1` は Windows用ビルドの印
- `code --version` は3行出力される（バージョン番号、コミットハッシュ、CPU種別）

#### ▼ もし `command not found` などのエラーが出たら:

- `node: command not found` → Node.jsが未インストール。1.2 章に戻ってインストール
- `git: command not found` → Gitが未インストール。4.2 章に戻ってインストール
- `code: command not found` → VS Codeのインストールはできているが、`code` コマンドのパスが通っていない。VS Codeのコマンドパレット（`Ctrl+Shift+P`）で「Shell Command: Install 'code' command in PATH」を実行

## 6.2 チェックリスト

以下のすべてにチェックが入れば、環境構築は完了です。

- ☐ Node.js がインストールされている（`node -v` で `v18.0.0` 以上が表示される）
- ☐ npm がインストールされている（`npm -v` でバージョンが表示される）
- ☐ Git がインストールされている（`git --version` でバージョンが表示される）
- ☐ Git の初期設定 が完了している（`git config --global user.name` で名前が表示される）
- ☐ VS Code がインストールされている（`code --version` でバージョンが表示される）
- ☐ VS Code の日本語化 が完了している（メニューが日本語で表示される）
- ☐ VS Code の必須拡張機能がインストールされている（ESLint, Prettier, React snippets）
- ☐ GitHub アカウント を作成済み

- [ ] ターミナル で基本コマンドが実行できる ( `pwd` , `ls` , `cd` など )

## 6.3 簡単なテスト

最後に、環境が正しく動作するかを実際に試してみましょう。

```
# 1. テスト用フォルダを作成
mkdir ~/Desktop/env-test
cd ~/Desktop/env-test

# 2. Node.js プロジェクトを初期化
npm init -y

# 3. 簡単な JavaScript ファイルを作成
cat > hello.js << 'EOF'
const message = "環境構築が完了しました！";
const nodeVersion = process.version;
const platform = process.platform;

console.log("=====");
console.log(message);
console.log(`Node.js バージョン: ${nodeVersion}`);
console.log(`プラットフォーム: ${platform}`);
console.log("=====");
console.log("");
console.log("次の章では、プロジェクトの作成を始めます。");
console.log("お疲れ様でした！");
EOF

# 4. 実行
node hello.js
```

以下のような出力が表示されれば成功です。

```
=====
環境構築が完了しました！
Node.js バージョン: v20.11.0
プラットフォーム: win32
=====

次の章では、プロジェクトの作成を始めます。
お疲れ様でした！
```

```
# 5. Git の動作確認
cd ~/Desktop/env-test
git init
git add .
git commit -m "環境テスト: 初回コミット"
git log --oneline

# 6. テストフォルダを削除 (任意)
cd ~/Desktop
rm -rf env-test
```

## 6.4 問題が解決しない場合

上記の手順で問題が解決しない場合は、以下を試してください。

1. PC を再起動する — 環境変数の変更が反映されていない可能性があります
2. ターミナルを新しく開き直す — 設定の再読み込みが必要な場合があります
3. ツールを再インストールする — インストール時にオプションを間違えた可能性があります
4. エラーメッセージで検索する — エラーメッセージをそのまま Google で検索すると、解決策が見つかることが多いです

## まとめ

この章では、以下のツールをインストールし、設定しました。

| ツール     | 用途               | 確認コマンド                      |
|---------|------------------|-----------------------------|
| Node.js | JavaScript の実行環境 | <code>node -v</code>        |
| npm     | パッケージマネージャー      | <code>npm -v</code>         |
| VS Code | コードエディタ          | <code>code --version</code> |
| Git     | バージョン管理          | <code>git --version</code>  |
| GitHub  | コードの共有・ホスティング    | ブラウザでログイン確認                 |

次の章では、これらのツールを使って実際に書籍管理アプリのプロジェクトを作成していきます。

 **ヒント:** 環境構築は最初の一回だけです。一度セットアップが完了すれば、今後の章ではコードを書くことに集中できます。ここまでお疲れ様でした！

次の章へ: [第2章 TypeScript の基礎](#) →